

# Practical Use Cases

Speaker companion

Seven tips for working with AI,  
one forecasting experiment to make them concrete

**How to use this document.** One page per slide in `jrc_section_v2_beamer.pdf` (slides 39–46 of the parent deck). Each page has: what's on screen, the prose to speak, the key numbers boxed so your eye can grab them, and the transition line into the next slide. Designed to be readable on a phone or second screen while presenting.

Total speaking budget: **~15 minutes**. Pace yourself: about **2 minutes per content slide**.

SLIDE 39 (divider)

## Practical Use Cases

---

### ON SCREEN

Large JRC-blue title “*Practical Use Cases*”, subtitle “*Seven tips, one forecasting experiment*”, your name underneath. No bullets, no figures.

#### Say this

*(Walk on, click to this slide, pause one beat.)*

“So far we’ve talked about *what* AI is and *where* it sits in our workflow. For the next fifteen minutes I want to make it concrete. I’m going to give you **seven practical tips** for working with AI on research code, and in the middle of those tips I’ll show you **one forecasting experiment** that quantifies why each tip actually matters. Same task, same model, three different prompts — and the result goes from useless to publishable.”

→ Next slide: Tips 1–5 as a single visual cluster. These are the habits the experiment will then test.

## SLIDE 40

## Seven tips for working with AI – part one

## ON SCREEN

Five JRC-blue tip cards: **Tips 1 & 2** on a wide top row, **Tips 3, 4, 5** on a narrower bottom row. No figures.

**Say this**

“Five tips first, then I’ll show you the experiment that proves them, then two more tips and a closing thought.”

**TIP 1 — Learn how to program, use your domain knowledge.**

“AI is a multiplier of what you already know. It cannot replace the domain knowledge of an economist, a physicist, an engineer. If you don’t know what *good* looks like for the task, no prompt will rescue you — you won’t be able to tell when the output is wrong. That’s the failure mode I see most often.”

**TIP 2 — Be as specific as humanly possible.**

“AI is only as good as the context you give it, and most people don’t give it enough. Three concrete moves: **(a)** paste the docs — API references, methodology papers, code examples — into the prompt or attach them as files; **(b)** use screenshots, much easier than describing a chart or a UI in words; **(c)** if you have a rough draft prompt, paste it back to the AI and say ‘Rewrite this using LLM best practices for a structured task description’ — keep what improves it, discard what doesn’t. Also: get the AI to search online, but *also* Google it yourself. Don’t make the agent guess what you already know how to look up.”

**TIP 3 — Smaller tasks beat big ones.**

“AI is good at small tasks, worse at big complex ones. Break a big task into smaller ones. And here’s the diagnostic: **if you can’t break it down, you don’t understand the problem well enough yet**. This isn’t an AI trick, it’s fundamental engineering — plan, decompose, then code.”

**TIP 4 — Tell the AI what you *don’t* want.**

“LLMs love to add things you didn’t ask for — extra abstractions, helper functions, defensive try/excepts. The fix is a three-section prompt pattern I use for almost everything: **TASK** — what you want done. **BACKGROUND** — context, constraints, data. **DO NOT** — explicit boundaries: don’t add a Flask wrapper, don’t refactor the parent function, don’t invent file paths. The DO NOT section is the one most people skip and it’s the one that saves the most time downstream.”

**TIP 5 — Tell the AI to remember.**

“You shouldn’t re-paste your project context every prompt. Two mechanisms. **AGENTS.md** is a file the agent reads automatically at the start of every session — put your tech stack, file layout, data sources, conventions, things the agent should never do. **Skills** live in `.claude/skills/` and codify reusable workflows — a named procedure the agent can invoke. We’ll use one in a minute. And the lazy starting point for AGENTS.md: ask the agent to analyse your repo and propose a v1, then edit.”

**Anchor facts**

- **Tip 4 pattern:** TASK / BACKGROUND / DO NOT — the DO NOT section is the load-bearing one.
- **Tip 5 files:** AGENTS.md for standing context · `.claude/skills/` for reusable procedures.
- **Time check:** aim to leave this slide around the 3-minute mark.

→ **Next slide:** The experiment. Same task, same model, three workspace setups — let’s see how much the five tips above actually move the needle.

## SLIDE 41

## The experiment

### ON SCREEN

Bulleted task description (forecast, data, agent, repo URL), then three side-by-side cards: **Level 1** · **Level 2** · **Level 3**. OPSD URL as small footnote at the bottom.

#### Say this

“Concrete setup. **Task**: forecast 168 hours — one week — of hourly German electricity load, the first week of January 2020. **Data**: Open Power System Data, ENTSO-E, 2015 to 2020, hourly, about 50,000 rows. **Agent**: Claude Code, Opus 4.7. Same agent in every run. Same data in every run. Code, prompts, and full results are on GitHub at the link on screen — feel free to clone it.”

“The three workspace setups model three different **user personae**, not three different models. The agent is constant — what changes is how much context and prompt sophistication the user brings.

**Level 1 — beginner**. Ten words. ‘Forecast first week of January 2020 German hourly electricity load.’ No AGENTS.md. This is what a researcher does on day one with Claude Code, no setup, no project context.

**Level 2 — average user**. Forty-six words. Names the data file, names the libraries, asks for a plot and an accuracy number. Has a seven-line AGENTS.md that points at the CSV. This is the average user after about a week — they’ve learned that AI assistants exist and they’re giving it slightly more.

**Level 3 — research-grade**. Sixteen hundred and seventy-three words. Structured TASK / BACKGROUND / DO NOT, asks for a six-model bake-off, held-out validation, prediction intervals. Backed by a 113-line AGENTS.md.”

“The point of this slide: prompt and workspace context scale *together*. A real beginner doesn’t have AGENTS.md set up; a pro does. The three rows of the headline aren’t just three different prompts, they’re three different *user personae*.”

#### Anchor facts

- **Same agent, same data, same task** in all three runs — only the human’s setup changes.
- Prompt word counts: **10 · 46 · 1,673**.
- AGENTS.md line counts: **0 · 7 · 113**.
- Repo URL is on screen if anyone wants to clone — don’t read it aloud.

→ Next slide: What ten words actually gets you.

## SLIDE 42

## Level 1 – what 10 words gets you

### ON SCREEN

The L1 prompt in italics at the top. Forecast plot: black actual line, single predicted line (seasonal-naive baseline). Caption underneath.

#### Say this

“Level 1. The prompt on screen is **ten words**: ‘Forecast first week of January 2020 German hourly electricity load.’ That’s the whole brief. No file path, no library hint, no accuracy requirement, no AGENTS.md.”

“What did the agent do? It picked three baselines, compared them on the last week of training data, and chose the one with the lowest error. **Seasonal-naive with a 364-day lag won — MAPE 10.76%**. Which honestly isn’t terrible for ten words of effort — the agent did something sensible. But look at what’s *missing*.”

“**No held-out validation set.** The model was evaluated on the same data it picked from. **No prediction intervals.** You don’t know whether that line is plus-or-minus five percent or plus-or-minus thirty. **No methodology document.** If a colleague asked you ‘what did this code do, and should I trust it’, you couldn’t answer.”

“That’s the L1 ceiling. Not because the agent is dumb — because we asked for ten words of effort and it gave us ten words of rigour.”

#### Anchor facts

- **Prompt:** 10 words, no AGENTS.md.
- **Winner:** seasonal-naive (lag-364d). **MAPE 10.76%**.
- **Missing:** held-out validation, prediction intervals, methodology document.
- Tip-4 link: this is what happens when there’s no DO NOT section — the agent picks the laziest defensible thing.

→ Next slide: Same task, slightly more prompt and a thin AGENTS.md — the average user.

## SLIDE 43

## Level 2 – average prompt + thin workspace

### ON SCREEN

L2 prompt in italics, AGENTS.md in greyed-out monospace underneath. Forecast plot: actual vs predicted, GradientBoostingRegressor. Caption.

#### Say this

“Level 2. The prompt now names the data file, names the libraries — pandas, matplotlib — and asks for a plot *and* an accuracy number. Forty-six words. The AGENTS.md is seven lines: tells the agent it’s a Python project, points at the CSV, says use pandas and matplotlib.”

“What changes? The agent picks a real model this time — **GradientBoostingRegressor** from scikit-learn. It engineers a handful of features by hand: hour of day, day of week, a 168-hour lag for weekly seasonality, an 8,760-hour lag for yearly. Reasonable engineering. **MAPE drops to 5.52%**. About half the error of L1.”

“But — and this is the key — **still no held-out validation. Still no prediction intervals. Still no methods document.** The number is better. The *rigour* hasn’t moved. We got lucky that the agent picked a decent model out of all the things it *could* have picked.”

“Pause on that. **‘We got lucky that it picked a decent model.’** That’s the L2 trap. You see a good number and you stop asking the question that matters: *would this hold up if I tested it properly?* L3 doesn’t have to be lucky.”

#### Anchor facts

- **Prompt:** 46 words, 7-line AGENTS.md.
- **Model:** GradientBoostingRegressor, hand-engineered features.
- **MAPE 5.52%** — roughly half of L1.
- **Still missing:** validation, intervals, methods. *Good number, no rigour.*

→ Next slide: Research-grade prompt. The number is only part of the story.

## SLIDE 44

## Level 3 – research-grade prompt

### ON SCREEN

Small grey caption summarising the prompt setup. Forecast plot with shaded prediction-interval bands around the LightGBM line. Two captions: the MAPE + interval coverage, then the L2/L3 contrast in lighter grey.

#### Say this

“Level 3. The prompt is sixteen hundred and seventy-three words and follows exactly the **TASK / BACKGROUND / DO NOT** pattern from Tip 4. It asks for a six-model bake-off, held-out validation, calibrated prediction intervals, and a methods document. Backed by a 113-line AGENTS.md.”

“The agent ran six models — naive baselines, gradient-boosted machines, LightGBM, a Prophet decomposition, and an ARIMA. It held out the last week before the test period as a validation set. It calibrated prediction intervals from the residuals on that validation set.”

“**LightGBM won. MAPE 3.43%.** About a third of L1’s error, about two-thirds of L2’s. But look at what’s underneath the line on screen — those are calibrated 80% and 95% prediction intervals. The **80% interval covered 79.8%** of the actuals; the **95% interval covered 92.9%**. Essentially nominal. The model isn’t just accurate, it knows how confident to be.”

“Read the grey line at the bottom: **L2 might have got lucky picking a decent model. L3 didn’t have to be lucky — it searched, validated, calibrated, and documented.** That’s the difference. That’s what those extra 1,600 words of prompt bought.”

#### Anchor facts

- **Prompt:** 1,673 words · **AGENTS.md:** 113 lines.
- **Winner:** LightGBM out of 6 models. **MAPE 3.43%.**
- **Coverage:** 80% PI □ 79.8%; 95% PI □ 92.9%. Essentially nominal.
- **Punchline:** *L2 got lucky. L3 didn’t have to.*

→ **Next slide:** The MAPE is only the visible part. What the rigorous prompt *actually* produced.

## SLIDE 45

## Some of what L3 actually produced

### ON SCREEN

A 3×2 gallery of small figures: six-model comparison · scoreboard (MAPE / RMSE / MAE) · per-day MAPE heatmap · winner residuals by hour and day · normalised feature importance · Prophet decomposition. Caption: *L1 produced 1 figure. L3 produced 8.*

#### Say this

“Quick gallery — I don’t want to dwell on individual plots, the point is the *breadth*.”

“Top row: **six-model comparison** on the same axes; **scoreboard** of MAPE / RMSE / MAE side by side; **per-day MAPE heatmap** so you can see which days the winner struggled on. Bottom row: **residuals** broken out by hour and by day-of-week — diagnostic for systematic bias; **feature importance** — and the punchline there is the 168-hour lag dominates at about 71% of total gain, which is intuitive: yesterday this time last week is the single best predictor of right now; and finally a **Prophet trend and seasonal decomposition** as a sanity check.”

“**L1 produced one figure. L3 produced eight.** That’s not because L3 is showing off. It’s because the prompt asked for the analytical breadth that makes the result trustworthy. The **headline isn’t the MAPE** — it’s everything around the MAPE.”

#### Anchor facts

- Don’t read the figures one by one — point and move. **Breadth** is the point.
- Feature importance: lag\_168h dominates at ~**71%**.
- Sound bite: *The headline isn’t the MAPE — it’s everything around the MAPE.*

→ **Next slide:** Two more tips to close, and then the thesis.



## SLIDE 46

## Seven tips (cont.) and the closing thesis

## ON SCREEN

Left column: TIP 6 (MCPs) and TIP 7 (verify). Right column: a JRC-blue “Closing thesis” box with three bold lines: *AI amplifies the habits you already have. Let it type, not think. Skip rigour □ AI amplifies that too.*

## Say this

### TIP 6 — MCPs extend what AI can do.

“MCP stands for **Model Context Protocol**. Plug-in tools the agent can invoke at runtime. The three I’d start with: **Context7** fetches up-to-date library documentation on demand — no more pasting docs into prompts. **Hugging Face MCP** searches models, datasets, papers. **GitHub MCP** reads repos, issues, and PRs — useful when you’re working across projects. Two-minute install each, pays for itself the first time.”

### TIP 7 — Always give AI a way to verify its work.

“AI should never just write code. It also needs a way to *prove* the code works. For code: **unit, integration, and regression tests**. For any model fitting: a **held-out validation set** — which is exactly what separated L3 from L1 and L2. For predictions: **calibration metrics, interval coverage, residual diagnostics**. And yes, you can have the AI write the tests too — but **read them**. AI-written tests can ‘pass’ because they test the wrong thing. A failing test that catches a real bug is worth ten passing tests that don’t.”

### Closing thesis.

*(Slow down here. This is the line you want them to walk out remembering.)*

“The seven tips aren’t really AI tips. They’re **good engineering habits**: be specific, break the problem down, write things down, build verification in. Let AI **type for you, not think for you**. If you bring those habits, AI multiplies your output substantially. If you don’t — if you skip the tests, skip the docs, skip the validation — **AI amplifies that too**. You’ll get incorrect outputs faster, or you’ll spend more time correcting them than you’d have spent doing the work yourself the first time around.”

“The forecasting experiment is one concrete instance of that. Same agent, same data — the rigour of the human is what moved MAPE from 10.76 to 3.43, *and* more importantly turned a single plot into a defensible methodology. Thanks.”

### Anchor facts

- **MCPs to mention by name:** Context7, Hugging Face, GitHub.
- **Tip 7 link-back:** the held-out validation set is literally what separated L3 from L1/L2.
- **Sound bites:** *Let AI type, not think.* · *AI amplifies the habits you already have — including the bad ones.*
- **Time check:** aim to land the closing line around the 14-minute mark so the next speaker has a clean handover.

End of practical use cases section. Hand over to Andres for *Risks & Infrastructure*.